

MONTE CARLO PREDICTION

RL → Learn from experience (episodes)

MC → uses actual returns, no model, every time whole episode

1. Introduction

- Monte Carlo (MC) methods learn from complete episodes of experience.
- In MC prediction, we estimate the value of states under a given policy π by averaging the actual returns observed after each visit to a state.
- It does not require a model of the environment.
- It is called on-policy prediction because it uses data generated by the policy itself.

2. Goal

- Learn an estimate $V_{\pi}(s)$ for every state s under a given policy π .

3. Return and Value Estimate

- Trajectory (episode) generated by following policy π : $S_0, A_0, S_1, A_1, \dots, S_{T-1}, A_{T-1}, S_T$
- Return from time t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$$

- $\gamma \in [0, 1]$: discount factor.

4. Intuition

- Each visit gives a sample of how good the state was.
- Average the returns over all visits to get the best estimate.

5. Example

Consider a small environment with reward $r = 0$ for all non-terminal transitions, and $r = +1$ when the episode terminates (reaches goal).

Suppose we generate the following episodes under policy π and visit state B multiple times:

Episode	Episode sequence (states)	Return from B (G_t)
1	B → C → Goal	$1 + 0 = 1$
2	A → B → C → Goal	$1 + 0 + 0 = 1$
3	B → D → Goal	$1 + 0 = 1$
4	A → B → C → Goal	$1 + 0 + 0 = 1$
5	B → C → Goal	$1 + 0 = 1$

Update for state B (at final step):

$$N(B) = 3, \quad V(B) = 0 + \frac{1}{3}(1-0) + \frac{1}{3}(1-0) + \frac{1}{3}(1-0)$$

$$= 1 \quad V(B) = 1$$

Key Idea :

Estimate the value of each state $s \in S$ by averaging the returns G obtained after every visit to s under the policy π .

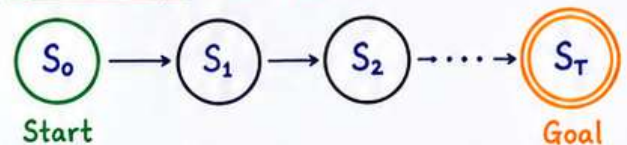
Why prediction?

- Evaluate a given policy π (not to improve it).
- Simple, model-free and easy to implement.

MC Prediction Algorithm (for policy π)

- Initialize $V(s) = 0, N(s) = 0$ for all $s \in S$
- Generate an episode $\tau = S_0, A_0, S_1, A_1, \dots, S_{T-1}, A_{T-1}, S_T$
- For each state s visited in the episode:
 - compute return G_t (from t to end)
 - $N(s) = N(s) + 1$
 - $V(s) \leftarrow V(s) + \frac{1}{N(s)} [G_t - V(s)]$

Episode Sample (one episode):



What happens during update?

- For each state on the episode, the return from that state to the end is computed.
- The value of that state is moved towards the observed return.
- Over many episodes, $V(s)$ converges to the true state value $V_{\pi}(s)$ (on average).

6. Properties and Observations

- Uses only complete episodes (no bootstrapping).
- Unbiased estimates (in expectation).
- Converges to the true value $V_{\pi}(s)$ if every state is visited infinitely often.
- Simple and easy to implement.

Key Takeaway

Monte Carlo prediction estimates the value of each state by averaging the actual returns observed after every visit to that state under a given policy π .
No model, just experience!



Monte Carlo Estimation of Action Values

1. INTRODUCTION

- Monte Carlo (MC) methods learn from complete episodes of experience.
- In MC Estimation of Action Values, the goal is to estimate the expected return (value) of taking an action in a state and then following policy π .
- It is used in action-value methods.
- No model of the environment is required.

KEY IDEA

Estimate the value of state-action pairs $Q^\pi(s, a)$ by averaging the actual returns obtained after taking action a in state s and following policy π until the end of the episode.

2. DEFINITIONS

- Episode:** A sequence $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$ that terminates at time T .
- Return from time t :** $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$
- Action-value function under policy π :**

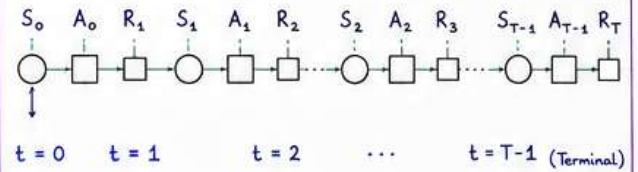
$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t | S_t = s, A_t = a]$$

(Expected return when action a is taken in state s and policy π is followed thereafter.)

3. MC ESTIMATION OF ACTION VALUES

- Initialize:** For all state-action pairs (s, a) ,
 $Q(s, a) = 0$, $N(s, a) = 0$
 where $N(s, a)$ counts how many times action a in state s has been taken.
- Generate an episode following policy π .
- For each time step t in the episode:
 - Compute return G_t from time t to the end:
 $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$
 - First-visit MC (commonly used):
 If the state-action pair (S_t, A_t) appears first at time t in the episode, then update:
 $N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$
 $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} [G_t - Q(S_t, A_t)]$
 (Incremental average)
- Repeat for many episodes.

EPISODE STRUCTURE (Example)



WHY FIRST-VISIT?

Using every occurrence of a pair in an episode (every-visit MC) also gives correct estimates, but first-visit is simpler and uses less data.

FORMULA SUMMARY

For a first visit of (s, a) at time t in an episode:

$$N(s, a) \leftarrow N(s, a) + 1$$

$$Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s, a)} [G_t - Q(s, a)]$$

Where,

$$G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k$$

is the return from time t .

4. EXAMPLE

Consider an episode ($\gamma = 1$ for simplicity):

t	0	1	2	3	4 (Terminal)
S_t	A	B	A	C	-
A_t	a_1	a_2	a_1	a_3	-
R_{t+1}	1	2	-1	3	-

Episode: $A \xrightarrow{a_1} B \xrightarrow{a_2} A \xrightarrow{a_1} C \xrightarrow{a_3} \text{Terminal}$

Step 1: Compute returns G_t (with $\gamma = 1$)

$$G_3 = R_4 = 3$$

$$G_2 = R_3 + R_4 = (-1) + 3 = 2$$

$$G_1 = R_2 + R_3 + R_4 = 2 + (-1) + 3 = 4$$

$$G_0 = R_1 + R_2 + R_3 + R_4 = 1 + 2 + (-1) + 3 = 5$$

Step 2: First-visit updates

$t = 0$, $(S_0, A_0) = (A, a_1)$ (first visit)

$$N(A, a_1) = 1$$

$$Q(A, a_1) = 0 + \frac{1}{1} (G_0 - 0) = 5$$

$t = 1$, $(S_1, A_1) = (B, a_2)$ (first visit)

$$N(B, a_2) = 1$$

$$Q(B, a_2) = 0 + \frac{1}{1} (G_1 - 0) = 4$$

$t = 2$, $(S_2, A_2) = (A, a_1)$ (already visited at $t=0 \rightarrow$ skip)

$t = 3$, $(S_3, A_3) = (C, a_3)$ (first visit)

$$N(C, a_3) = 1$$

$$Q(C, a_3) = 0 + \frac{1}{1} (G_3 - 0) = 3$$

KEY POINTS

- Estimates the expected return of taking an action in a state and following policy π .
- Uses complete episodes (no bootstrapping).
- First-visit is commonly used.
- Estimates converge to true $Q^\pi(s, a)$ as the number of episodes $\rightarrow \infty$.
- Unbiased estimates but may have high variance.

After this episode:

$$Q(A, a_1) = 5, \quad Q(B, a_2) = 4, \quad Q(C, a_3) = 3$$

Monte Carlo Control and Implementation

1. INTRODUCTION

- Monte Carlo (MC) Control methods learn the optimal policy directly from complete episodes of experience.
- The goal is to find a policy π^* that maximizes the expected return.
- MC Control does not require a model of the environment.
- It uses exploration (e.g., ϵ -greedy) to try actions and improve the policy.

2. DEFINITIONS

- **Episode**: A sequence $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$ that terminates at time T .
- Return from time t : $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$
- Action-value function under policy π :
$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t | S_t = s, A_t = a]$$

(Expected return starting from state s , taking action a , and following π thereafter.)
- Goal of MC Control:
Find optimal action-value function $Q^*(s, a)$ and the optimal policy π^* such that
$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

3. MONTE CARLO CONTROL (On-policy, ϵ -greedy)

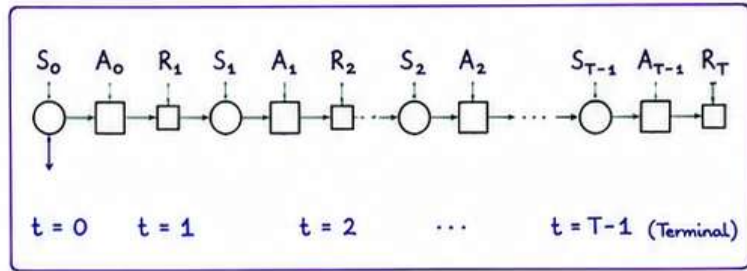
- ① Initialize: For all state-action pairs (s, a) ,
 $Q(s, a) = 0$ and $N(s, a) = 0$
(where $N(s, a)$ counts how many times (s, a) has occurred.)
- ② Generate an episode using an ϵ -greedy policy derived from Q .
- ③ For each time step t in the episode:
 - Compute return G_t from time t to the end.
 - First-visit MC update (commonly used):
If the pair (S_t, A_t) appears first at time t in the episode, then update:
$$N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$$
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} [G_t - Q(S_t, A_t)]$$

(Incremental average)
- ④ Improve policy: Make the policy ϵ -greedy with respect to the updated Q (i.e., take the action with highest Q with probability $1 - \epsilon$, otherwise explore).
- ⑤ Repeat for many episodes until Q converges.

KEY POINTS

- MC Control learns both action values $Q^*(s, a)$ and the optimal policy π^* .
- Uses complete episodes (no bootstrapping).
- On-policy method (the behavior policy is the same as the target/improved policy).
- Converges to optimal policy as the number of episodes $\rightarrow \infty$ if all state-action pairs are explored infinitely often.
- ϵ -greedy exploration ensures exploration and convergence.

EPISODE STRUCTURE (Example)



POLICY IMPROVEMENT (ϵ -greedy)

For a state s :

- With probability $1 - \epsilon$: choose $a^* = \arg \max_a Q(s, a)$
- With probability ϵ : choose a random action from the available actions.

4. IMPLEMENTATION (PSEUDOCODE)

Input: Number of episodes N , discount factor γ , exploration rate ϵ
Output: Optimal action-value function $Q^*(s, a)$
Initialize $Q(s, a) = 0$ and $N(s, a) = 0$ for all (s, a)
For episode = 1 to N :
 Generate an episode using ϵ -greedy policy based on Q
 Get episode: $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$
 For $t = 0$ to $T-1$:
 Compute return $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$
 If (S_t, A_t) is a first visit in the episode:
 $N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$
 $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} [G_t - Q(S_t, A_t)]$
 Improve policy to be ϵ -greedy with respect to Q
Return $Q(s, a)$ and the greedy policy $\pi^*(s) = \arg \max_a Q(s, a)$

5. EXAMPLE (Very Small Example)

Consider a simple episode ($\gamma = 1$):

States visited: $A \rightarrow B \rightarrow C$ (Terminal)

Rewards received after each action: $+1, +2, -1$

Episode (time sequence):

$S_0 = A, A_0 = a_1, R_1 = +1, S_1 = B, A_1 = a_2, R_2 = +2,$

$S_2 = A, A_2 = a_1, R_3 = -1, S_3 = C$ (Terminal)

Returns:

$$G_2 = R_3 = -1$$

$$G_1 = R_2 + R_3 = 2 + (-1) = 1$$

$$G_0 = R_1 + R_2 + R_3 = 1 + 2 + (-1) = 2$$

After first episode (using first-visit updates):

$$Q(A, a_1) = 2 \quad (\text{from time 0 only})$$

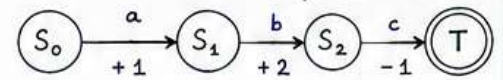
$$Q(B, a_2) = 1$$

$Q(A, a_1)$ (second visit at $t = 2$ not used in first-visit update).

MONTE CARLO - SOLVED PROBLEM

Problem: Consider the following small episodic MDP. Start state is S_0 and terminal state is T . Rewards are on transitions.

The behaviour policy π takes actions with equal probability. Use Monte Carlo (First-Visit) prediction to estimate the value function $V^\pi(s)$ for states S_0, S_1, S_2 .



All other actions keep the agent in the same state with reward 0.

1. EPISODES GENERATED

We generate 5 episodes by following policy π (all actions equally likely).

Episode	Sequence of states and rewards	Returns (G_t)
①	$S_0 \xrightarrow{a/+1} S_1 \xrightarrow{b/+2} S_2 \xrightarrow{c/-1} T$	$G_0 = 1 + 2 - 1 = 2, G_1 = 2 - 1 = 1, G_2 = -1$
②	$S_0 \xrightarrow{a/+1} S_1 \xrightarrow{b/+2} S_2 \xrightarrow{c/-1} T$	$2, 1, -1$
③	$S_0 \xrightarrow{a/+1} S_1 \xrightarrow{(\text{stay } 0)} S_1 \xrightarrow{b/+2} S_2 \xrightarrow{c/-1} T$	$G_0 = 1 + 0 + 2 - 1 = 2, G_1 = 0 + 2 - 1 = 1, G_2 = 2 - 1 = 1, G_3 = -1$
④	$S_0 \xrightarrow{(\text{stay } 0)} S_0 \xrightarrow{a/+1} S_1 \xrightarrow{b/+2} S_2 \xrightarrow{c/-1} T$	$G_0 = 0 + 1 + 2 - 1 = 2, G_1 = 1 + 2 - 1 = 2, G_2 = 2 - 1 = 1, G_3 = -1$
⑤	$S_0 \xrightarrow{a/+1} S_1 \xrightarrow{(\text{stay } 0)} S_1 \xrightarrow{b/+2} S_2 \xrightarrow{(\text{stay } 0)} S_2 \xrightarrow{c/-1} T$	$G_0 = 1 + 0 + 2 + 0 - 1 = 2, G_1 = 0 + 2 + 0 - 1 = 1, G_2 = 2 + 0 - 1 = 1, G_3 = 0 - 1 = -1$

2. FIRST-VISIT RETURNS

For each episode, record the return the first time a state is visited.

Episode	First visit of S_0	First visit of S_1	First visit of S_2
①	$G_0 = 2$	$G_1 = 1$	$G_2 = -1$
②	$G_0 = 2$	$G_1 = 1$	$G_2 = -1$
③	$G_0 = 2$	$G_1 = 1$	$G_2 = 1$
④	$G_0 = 2$	$G_1 = 2$	$G_2 = 1$
⑤	$G_0 = 2$	$G_1 = 1$	$G_2 = 1$

3. MONTE CARLO ESTIMATE (FIRST-VISIT)

Estimate value by averaging first-visit returns.

- For S_0 : Returns = $\{2, 2, 2, 2, 2\}$
 $V^\pi(S_0) = (2+2+2+2+2)/5 = 2.0$
- For S_1 : Returns = $\{1, 1, 1, 2, 1\}$
 $V^\pi(S_1) = (1+1+1+2+1)/5 = \frac{6}{5} = 1.2$
- For S_2 : Returns = $\{-1, -1, 1, 1, 1\}$
 $V^\pi(S_2) = (-1-1+1+1+1)/5 = \frac{1}{5} = 0.2$

Final Value Estimates (Monte Carlo First-Visit)

$$V^\pi(S_0) = 2.0, V^\pi(S_1) = 1.2, V^\pi(S_2) = 0.2$$

$$(V^\pi(T) = 0 \text{ by definition})$$

4. SUMMARY TABLE

State	Number of First Visits	Estimated $V^\pi(s)$
S_0	5	2.0
S_1	5	1.2
S_2	5	0.2
T	—	0

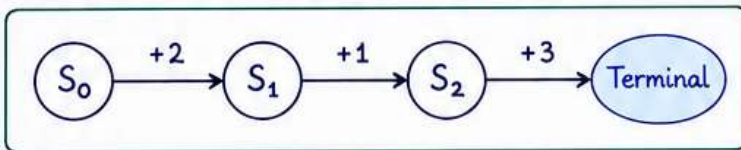
5. REMARKS

- We used First-Visit Monte Carlo prediction.
- As the number of episodes increases, the estimates converge to the true values under π .
- No model of the environment was needed.
- All returns are obtained from complete episodes.

Monte Carlo Prediction - Solved Example

Problem :

Consider the following episodic environment :



Discount factor : $\gamma = 1$

The following episodes are generated by following policy π .

Episodes :

Episode	State sequence	Rewards
1	$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow T$	+2 , +1 , +3
2	$S_0 \rightarrow S_1 \rightarrow S_1 \rightarrow S_2 \rightarrow T$	+2 , 0 , +1 , +3
3	$S_0 \rightarrow S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow T$	0 , +2 , +1 , +3
4	$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_2 \rightarrow T$	+2 , +1 , 0 , +3

Task :

Using First-Visit Monte Carlo Prediction, estimate the state values $V(S_0)$, $V(S_1)$ and $V(S_2)$.

1. Returns for each episode ($\gamma = 1$)

Episode 1 : $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow T$ (rewards: 2, 1, 3)

$$G(S_0) = 2 + 1 + 3 = 6$$

$$G(S_1) = 1 + 3 = 4$$

$$G(S_2) = 3$$

Episode 2 : $S_0 \rightarrow S_1 \rightarrow S_1 \rightarrow S_2 \rightarrow T$ (rewards: 2, 0, 1, 3)

$$G(S_0) = 2 + 0 + 1 + 3 = 6$$

$$G(S_1) = 0 + 1 + 3 = 4 \quad (\text{first visit of } S_1)$$

$$G(S_2) = 3$$

Episode 3 : $S_0 \rightarrow S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow T$ (rewards: 0, 2, 1, 3)

$$G(S_0) = 0 + 2 + 1 + 3 = 6 \quad (\text{first visit of } S_0)$$

$$G(S_1) = 1 + 3 = 4$$

$$G(S_2) = 3$$

Episode 4 : $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_2 \rightarrow T$ (rewards: 2, 1, 0, 3)

$$G(S_0) = 2 + 1 + 0 + 3 = 6$$

$$G(S_1) = 1 + 0 + 3 = 4$$

$$G(S_2) = 0 + 3 = 3 \quad (\text{first visit of } S_2, \text{ ignore second})$$

Monte Carlo Prediction (Theory)

- Estimates the state value function $V^*(s)$ by averaging the returns obtained from complete episodes.
- Uses no model of the environment.
- First-visit MC uses the return obtained from the first time a state is visited in an episode.

Update Rule (Incremental form)

$$V(s) \leftarrow V(s) + \frac{1}{N(s)} [G_t - V(s)]$$

where,

G_t - return from the first visit of state s

$N(s)$ - number of first visits to state s

2. First-Visit Returns Table

Episode	First-visit return		
	S_0	S_1	S_2
1	6	4	3
2	6	4	3
3	6	4	3
4	6	4	3

3. Monte Carlo Estimates

Average the first-visit returns for each state.

$$V(S_0) = \frac{6 + 6 + 6 + 6}{4} = 6.0$$

$$V(S_1) = \frac{4 + 4 + 4 + 4}{4} = 4.0$$

$$V(S_2) = \frac{3 + 3 + 3 + 3}{4} = 3.0$$

Final Result :

$$V(S_0) = 6.0 \quad V(S_1) = 4.0 \quad V(S_2) = 3.0$$

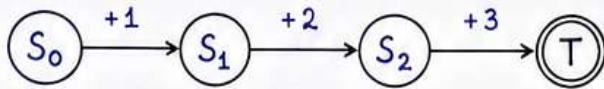
4. Key Points

- Monte Carlo Prediction estimates state values from complete episodes (no bootstrapping).
- Simple and unbiased but requires many episodes for accurate estimates.
- Works well for episodic tasks and is a basis for Monte Carlo Control.

MONTE CARLO - SOLVED EXAMPLE (FIRST-VISIT PREDICTION)

PROBLEM STATEMENT :

Consider the following episodic environment.



An episode starts from S_0 and ends at terminal state T .

Rewards are received on transitions.

Use Monte Carlo (First-Visit) prediction to estimate the value function $V^\pi(s)$ for states S_0 , S_1 and S_2 .

EPISODES GENERATED BY POLICY π :

Episode 1 : $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow T$

Rewards : +1 , +2 , +3

Episode 2 : $S_0 \rightarrow S_1 \rightarrow S_1 \rightarrow S_2 \rightarrow T$

Rewards : +1 , 0 , +2 , +3

Episode 3 : $S_0 \rightarrow S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow T$

Rewards : 0 , +1 , +2 , +3

Episode 4 : $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_2 \rightarrow T$

Rewards : +1 , +2 , 0 , +3

Discount factor $\gamma = 1$

STEP 1 : COMPUTE RETURNS G_t FOR EACH TIME STEP

G_t = sum of all future rewards from time t till the end of the episode ($\gamma = 1$).

Episode 1

Time t	State	Reward	Return G_t
0	S_0	+1	$1+2+3=6$
1	S_1	+2	$2+3=5$
2	S_2	+3	3

Sequence : $S_0 \ S_1 \ S_2 \ T$
 Rewards : +1 +2 +3

Episode 2

Time t	State	Reward	Return G_t
0	S_0	+1	$1+0+2+3=6$
1	S_1	0	$0+2+3=5$
2	S_1	+2	$2+3=5$
3	S_2	+3	3

Sequence : $S_0 \ S_1 \ S_1 \ S_2 \ T$
 Rewards : +1 0 +2 +3

Episode 3

Time t	State	Reward	Return G_t
0	S_0	0	$0+1+2+3=6$
1	S_0	+1	$1+2+3=6$
2	S_1	+2	$2+3=5$
3	S_2	+3	3

Sequence : $S_0 \ S_0 \ S_1 \ S_2 \ T$
 Rewards : 0 +1 +2 +3

Episode 4

Time t	State	Reward	Return G_t
0	S_0	+1	$1+2+0+3=6$
1	S_1	+2	$2+0+3=5$
2	S_2	0	$0+3=3$
3	S_2	+3	3

Sequence : $S_0 \ S_1 \ S_2 \ S_2 \ T$
 Rewards : +1 +2 0 +3

STEP 2 : COLLECT FIRST-VISIT RETURNS

For each episode, take the return only the first time the state is visited in that episode.

Episode	First-visit return of S_0	First-visit return of S_1	First-visit return of S_2
1	6	5	3
2	6	5 (first visit at $t=1$)	3
3	6 (first visit at $t=0$)	5	3
4	6	5	3

Note :

In Episode 2, state S_1 appears twice. We take return (5) from its first visit ($t=1$).

In Episode 3, state S_0 appears twice. We take return (6) from its first visit ($t=0$).

STEP 3 : ESTIMATE VALUE FUNCTION $V^\pi(s)$

Average the first-visit returns for each state.

For state S_0 :

Returns = {6, 6, 6, 6}

$$V^\pi(S_0) = \frac{6+6+6+6}{4} = \frac{24}{4} = 6$$

For state S_1 :

Returns = {5, 5, 5, 5}

$$V^\pi(S_1) = \frac{5+5+5+5}{4} = \frac{20}{4} = 5$$

For state S_2 :

Returns = {3, 3, 3, 3}

$$V^\pi(S_2) = \frac{3+3+3+3}{4} = \frac{12}{4} = 3$$

STEP 4 : FINAL VALUE FUNCTION

$$V^\pi(S_0) = 6, \quad V^\pi(S_1) = 5, \quad V^\pi(S_2) = 3$$

$$(V^\pi(T) = 0 \text{ by definition})$$

SUMMARY :

- We used First-Visit Monte Carlo Prediction.
- Returns are computed from the end of each episode.
- For each state, we used the return from its first visit in each episode.
- The value of each state is the average of those first-visit returns.

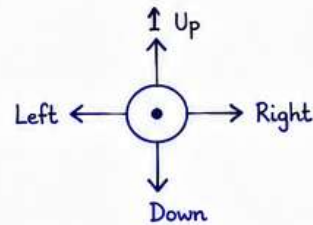
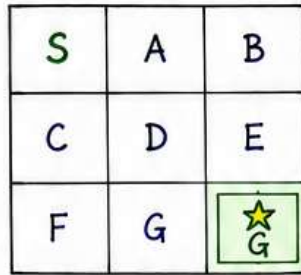
Answer : $V^\pi(S_0) = 6, \quad V^\pi(S_1) = 5, \quad V^\pi(S_2) = 3$

MONTE CARLO - GRID WORLD EXAMPLE

First-Visit Prediction

1. PROBLEM STATEMENT

- We have a 3x3 Grid World.
- Start state : S (top-left)
- Goal state : G (bottom-right)
- Actions : $\uparrow, \downarrow, \leftarrow, \rightarrow$
- Reward : -1 for each move
0 when goal G is reached



Goal State

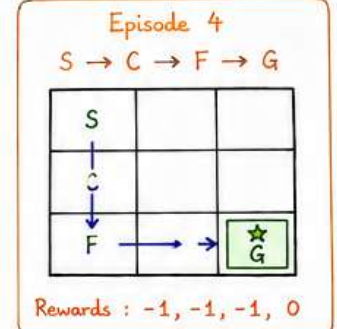
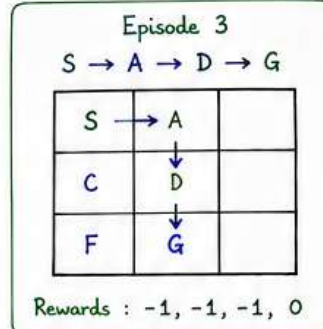
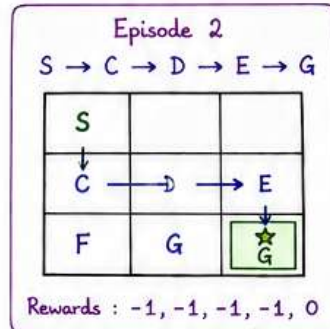
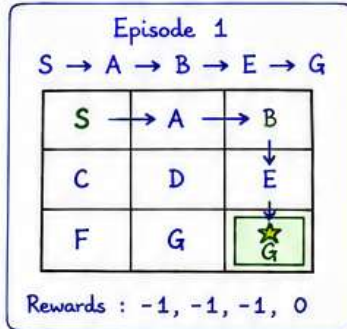
★ G

Reward = 0

Other States

Reward = -1
per move

2. EPISODES GENERATED BY RANDOM POLICY



3. STEP 1 - COMPUTE RETURNS G_t FOR EACH TIME STEP ($\gamma = 1$)

Episode 1			
t	State	Reward	Return G_t
0	S	-1	-3
1	A	-1	-2
2	B	-1	-1
3	E	-1	0
4	G	0	0

Sequence : S A B E G
Rewards : -1 -1 -1 -1 0

Episode 2			
t	State	Reward	Return G_t
0	S	-1	-4
1	C	-1	-3
2	D	-1	-2
3	E	-1	-1
4	G	0	0

Sequence : S C D E G
Rewards : -1 -1 -1 -1 0

Episode 3			
t	State	Reward	Return G_t
0	S	-1	-3
1	A	-1	-2
2	D	-1	-1
3	G	0	0

Sequence : S A D G
Rewards : -1 -1 -1 0

Episode 4			
t	State	Reward	Return G_t
0	S	-1	-3
1	C	-1	-2
2	F	-1	-1
3	G	0	0

Sequence : S C F G
Rewards : -1 -1 -1 0

4. STEP 2 - COLLECT FIRST-VISIT RETURNS

State	First-visit returns from each episode.				All first-visit returns
	Ep. 1	Ep. 2	Ep. 3	Ep. 4	
S	-3	-4	-3	-3	{-3, -4, -3, -3}
A	-2	—	-2	—	{-2, -2}
B	-1	—	—	—	{-1}
C	—	-3	—	-2	{-3, -2}
D	—	-2	-1	—	{-2, -1}
E	0	-1	—	—	{0, -1}
F	—	—	—	-1	{-1}
G (Goal)	0	0	0	0	{0.}

Notes :

- We take return only the first time a state is visited in an episode.
- Some states do not appear in some episodes.
- G has return 0 always.

5. STEP 3 - ESTIMATE VALUE FUNCTION $V^\pi(s)$

Average the first-visit returns for each state.

$$V^\pi(S) = \frac{-3 + (-4) + (-3) + (-3)}{4} = \frac{-13}{4} = -3.25$$

$$V^\pi(A) = \frac{-2 + (-2)}{2} = \frac{-4}{2} = -2$$

$$V^\pi(B) = \frac{-1}{1} = -1$$

$$V^\pi(C) = \frac{-3 + (-2)}{2} = \frac{-5}{2} = -2.5$$

$$V^\pi(D) = \frac{-2 + (-1)}{2} = \frac{-3}{2} = -1.5$$

$$V^\pi(E) = \frac{0 + (-1)}{2} = \frac{-1}{2} = -0.5$$

$$V^\pi(F) = \frac{-1}{1} = -1$$

$$V^\pi(G) = \frac{0}{1} = 0$$

6. FINAL VALUE FUNCTION $V^\pi(s)$

S	A	B
-3.25	-2	-1
C	D	E
-2.5	-1.5	-0.5
F	G	G (Goal)
-1	0	0 ★

Higher
Better
Lower
Worse

7. SUMMARY



Monte Carlo (First-Visit) uses experience (episodes) to estimate how good each state is under a policy.

Learn from Experience
(step by step)

TEMPORAL - DIFFERENCE (TD) LEARNING

No need of a model of the environment!

1. WHAT IS TD LEARNING ?

- TD Learning is a combination of Monte Carlo methods and Dynamic Programming.
- It learns **directly from interaction** with the environment.
- Updates estimates after **every step** (or transition) using other learned estimates.
- Can learn **online** (incrementally) without waiting for the end of an episode.

MC vs DP vs TD

	Monte Carlo	TD Learning	Dynamic Programming
Learns from	Complete episodes	Every step	Known model
Bootstrapping	No	Yes	Yes
Update frequency	End of episode	Every step	Synchronous (backup all states)
Example	MC Prediction	TD(0), SARSA, Q-learning	Policy Evaluation, Value Iteration

2. TD PREDICTION (TD(0))

Goal: Estimate the state value function $V^*(s)$ for a given policy π .

TD(0) Update Rule:

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t$$

Old estimate Step-size ($0 < \alpha \leq 1$) TD Error

TD Error (Difference):

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

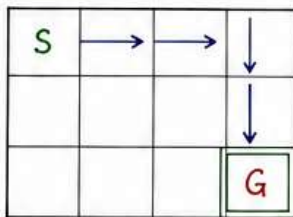
Reward received Expected (estimated) value of next state Current estimate of state S_t



Intuition:

We adjust the current estimate $V(S_t)$ by a fraction (α) of the difference between what we **actually observed** ($R_{t+1} + \gamma V(S_{t+1})$) and what we **expected** ($V(S_t)$).

3. EXAMPLE



S: Start state
G: Goal state
→: Possible path

Reward = -1 for each move,
0 when goal is reached.

Episode (one sample path):

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

Discount factor $\gamma = 1$

Step-size $\alpha = 0.5$

Initialize $V(s) = 0$
for all states.

t	Transition	R_{t+1}	Old $V(S_t)$	TD Error $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$	New $V(S_t) = V(S_t) + \alpha \delta_t$
0	$S \rightarrow A$	-1	0	$-1 + 0 - 0 = -1$	$0 + 0.5(-1) = -0.5$
1	$A \rightarrow B$	-1	0	$-1 + 0 - 0 = -1$	-0.5
2	$B \rightarrow C$	-1	0	$-1 + 0 - 0 = -1$	-0.5
3	$C \rightarrow G$	0	0	$0 + 0 - 0 = 0$	0

With more episodes, the values converge to the true values.

4. ADVANTAGES OF TD LEARNING



Online Learning

Updates after every step. No need to wait for the end of an episode.



Efficient

Uses bootstrapping → faster learning than Monte Carlo.



Works with incomplete episodes

Can learn even if the episode does not end.



Model-free

Does not require the transition probabilities of the environment.



Lower Variance

Usually has lower variance than Monte Carlo methods.

5. OPTIMALITY OF TD(0)

Consider TD(0) for policy evaluation with a fixed policy π .

Under the assumptions:

✓ Finite Markov chain

✓ $0 \leq \alpha_t \leq 1$, $\sum_{t=0}^{\infty} \alpha_t = \infty$, $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$.

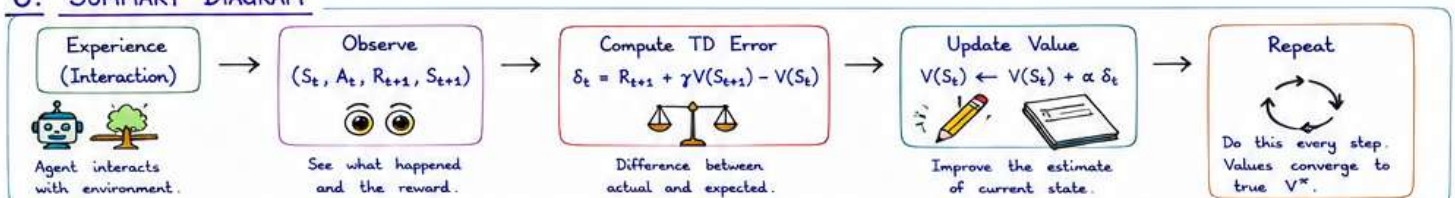
These are called Robbins-Monro conditions.

Then, TD(0) is "optimal" in the sense that:

- 1 It converges to the true value function V^* with probability 1.
- 2 Among all possible incremental methods that use the same information (R_{t+1} , S_{t+1}), TD(0) converges with minimum mean squared error.

Key Point: TD(0) converges to the true value and is statistically more efficient than other incremental methods. ★

6. SUMMARY DIAGRAM



In Short: ★

TD Learning learns from experience at every step using bootstrapping. It is fast, online, model-free and provably optimal (TD(0)).

Common TD Methods:

- TD(0) (prediction)
- SARSA, Q-learning (control)

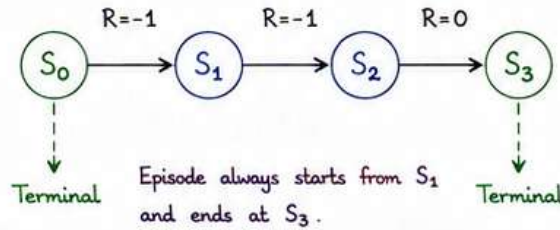
TEMPORAL-DIFFERENCE (TD) LEARNING

Learn from experience step by step!

SOLVED EXAMPLE USING TD(0) PREDICTION

1. PROBLEM SETUP

- We have a simple chain of states.
- S_0 and S_3 are terminal states.
- Rewards are received on transitions.
- Goal: Predict the state values $V(s)$ using TD(0) prediction.



Parameters

- Discount factor $\gamma = 1$
- Step-size $\alpha = 0.5$
- Initial values $V(S_1)=0, V(S_2)=0$
- Terminal values $V(S_0)=0, V(S_3)=0$

2. TD(0) UPDATE RULE

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t$$

TD error (Difference)

Old estimate of current state

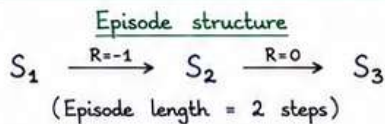
Step-size

Estimated value of next state

Reward received

We adjust $V(S_t)$ towards the target $[R_{t+1} + \gamma V(S_{t+1})]$ by a fraction α of the difference.

3. EPISODES (SAMPLE EXPERIENCE)



Since the environment is deterministic, every episode is the same:

$S_1 \rightarrow S_2 \rightarrow S_3$

Rewards

$R(S_1 \rightarrow S_2) = -1$

$R(S_2 \rightarrow S_3) = 0$

4. STEP-BY-STEP TD(0) UPDATES

Episode	Step (t)	Transition	R_{t+1}	Old $V(S_t)$	$V(S_{t+1})$	Target $R_{t+1} + \gamma V(S_{t+1})$	δ_t (Difference)	Updated $V(S_t) = V(S_t) + \alpha \delta_t$	
								$V(S_1)$	$V(S_2)$
1	0	$S_1 \rightarrow S_2$	-1	0	0	$-1 + 1 \times 0 = -1$	$-1 - 0 = -1$	$0 + 0.5(-1) = -0.5$	0
	1	$S_2 \rightarrow S_3$	0	0	0	$0 + 1 \times 0 = 0$	$0 - 0 = 0$	-0.5	$0 + 0.5(0) = 0$
2	0	$S_1 \rightarrow S_2$	-1	-0.5	0	$-1 + 1 \times 0 = -1$	$-1 - (-0.5) = -0.5$	$-0.5 + 0.5(-0.5) = -0.75$	0
	1	$S_2 \rightarrow S_3$	0	0	0	$0 + 1 \times 0 = 0$	$0 - 0 = 0$	-0.75	0
3	0	$S_1 \rightarrow S_2$	-1	-0.75	0	$-1 + 1 \times 0 = -1$	$-1 - (-0.75) = -0.25$	$-0.75 + 0.5(-0.25) = -0.875$	0
	1	$S_2 \rightarrow S_3$	0	0	0	$0 + 1 \times 0 = 0$	$0 - 0 = 0$	-0.875	0
4	0	$S_1 \rightarrow S_2$	-1	-0.875	0	$-1 + 1 \times 0 = -1$	$-1 - (-0.875) = -0.125$	$-0.875 + 0.5(-0.125) = -0.9375$	0
	1	$S_2 \rightarrow S_3$	0	0	0	$0 + 1 \times 0 = 0$	$0 - 0 = 0$	-0.9375	0

We can see $V(S_1)$ is moving towards -1, and $V(S_2)$ stays at 0 (true values).

5. TRUE VALUE FUNCTION (for reference)

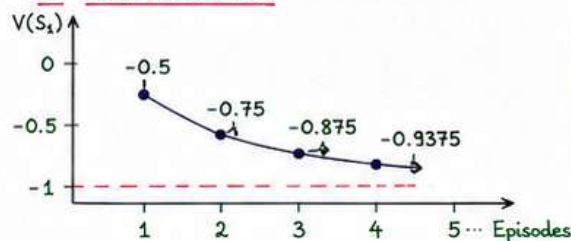
Using DP, the true state values are:

$$V^*(S_1) = -1$$

$$V^*(S_2) = 0$$

(Terminal states are 0)

6. CONVERGENCE



$V(S_1)$ converges to -1
 $V(S_2)$ already equals 0
 (both are the true values).

7. SUMMARY

- ✓ TD(0) prediction learns the value of states from experience one step at a time.
- ✓ It uses the next state's estimate to update the current state (bootstrapping).
- ✓ No model of the environment is required.
- ✓ With more episodes, the estimates converge to the true values.

Key Idea

TD(0) = Monte Carlo + DP
 → learns from experience using bootstrapping.

Learn from experience!

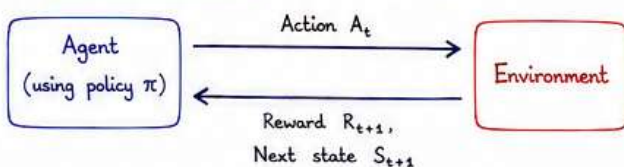
TD-SARSA (On-Policy TD Control)

On-policy: use the same policy to learn and act.

1. WHAT IS TD-SARSA?

- TD-SARSA is an **on-policy** Temporal-Difference (TD) control algorithm.
- It learns the action-value function $Q^{\pi}(s,a)$ for the policy π and improves it over time.
- "On-policy" means the agent learns from the experience it generates using its current policy.
- It updates the value of the current state-action pair using the next action **actually** taken.

2. HOW IT WORKS? (At each step)



- In state S_t , agent takes action A_t using policy π .
- Environment returns reward R_{t+1} and next state S_{t+1} .
- Agent selects next action A_{t+1} in S_{t+1} using same policy π .
- Update $Q(S_t, A_t)$ towards the target.



We learn and act using the same policy π .

3. TD-SARSA UPDATE RULE

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Old estimate $\uparrow$ Step-size $\uparrow$ ($0 < \alpha \leq 1$) Reward received $\uparrow$ Value of next state-action (using next action actually taken A_{t+1}) $\uparrow$ TD Error $\uparrow$ Current estimate $\uparrow$

★ Key Point.

We use A_{t+1} actually taken by the current policy. That's why it is **On-Policy**.

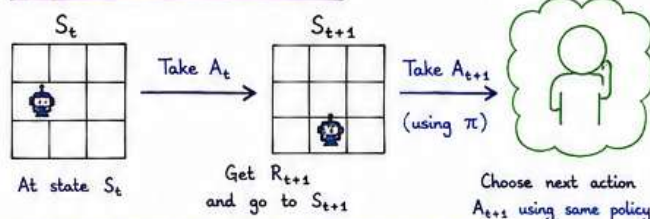
If $\gamma = 0$, only immediate reward is considered.

If $\alpha = 1$, we fully replace the estimate.

4. ALGORITHM (TD-SARSA)

- Initialize $Q(s,a)$ arbitrarily for all (s,a) .
- Choose a policy π (e.g., ϵ -greedy) and step-size α .
- Initialize S (start state).
- Choose A from S using policy π .
- Repeat (for each step of episode):
 - Take action A , observe R, S' .
 - Choose A' from S' using policy π .
 - Update: $Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma Q(S',A') - Q(S,A)]$
 - $S \leftarrow S', A \leftarrow A'$.
- Until episode terminates.

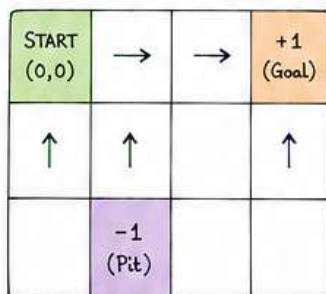
5. VISUAL FLOW (ONE STEP)



Properties

- ★ On-policy control method.
- ★ Combines ideas of Monte-Carlo (uses experience) and Dynamic Programming (bootstrapping).
- ★ Updates at every step (online learning).
- ★ Converges to Q^{π} under suitable conditions.

6. SOLVED EXAMPLE (GRID WORLD)



Arrows show available actions ($\uparrow, \downarrow, \leftarrow, \rightarrow$)

Assume:

- Reward = +1 at Goal, -1 at Pit, 0 otherwise.
- Episode ends at Goal or Pit.
- $\gamma = 1, \alpha = 0.5$
- Policy: ϵ -greedy (here we follow the shown actions)

Episode (following the arrows)

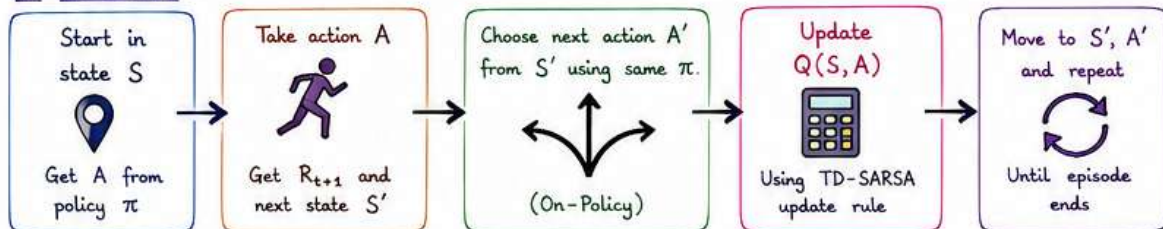
(0,0) $\rightarrow$ (0,1) $\rightarrow$ (0,2) $\rightarrow$ (0,3)
 START Right Right Right GOAL (+1)

We will update $Q(s,a)$ at each step using the next action actually taken.

Step t	S_t	A_t	R_{t+1}	S_{t+1}	A_{t+1}	TD Target $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$	Old $Q(S_t, A_t)$	Updated $Q(S_t, A_t)$
1	(0,0)	Right	0	(0,1)	Right	$0 + 1 \cdot Q(0,1, \text{Right}) = 0$	0	0
2	(0,1)	Right	0	(0,2)	Right	$0 + 1 \cdot Q(0,2, \text{Right}) = 0$	0	0
3	(0,2)	Right	+1	(0,3)	(terminal)	$1 + 1 \cdot 0 = 1$	0	$0 + 0.5 \cdot (1 - 0) = 0.5$

All other $Q(s,a)$ remain 0 until visited and updated.
 With more episodes, Q values will propagate back and improve.

7. IN SHORT



Takeaway ★

TD-SARSA learns action values from experience step by step while following the same policy. It is simple, online, and effective for many reinforcement learning problems.

Learn from experience!

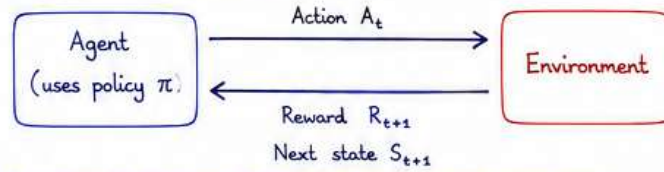
TD-SARSA (ON-POLICY TD CONTROL)

On-policy means we learn and act using the same policy.

1. WHAT IS TD-SARSA ?

- TD-SARSA is an on-policy Temporal-Difference (TD) control algorithm.
- It learns the action-value function $Q^\pi(s, a)$ for the policy π that is used to generate the episodes.
- It updates the value of the current state-action pair using the next action actually taken by the current policy.

2. HOW IT WORKS ? (Idea)



At each step, the agent:

- 1) Takes action A_t using policy π in state S_t
- 2) Receives reward R_{t+1} and next state S_{t+1}
- 3) Chooses next action A_{t+1} using the same policy π
- 4) Updates $Q(S_t, A_t)$ towards the target.

3. TD-SARSA UPDATE RULE

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Old estimate $\uparrow$ Step-size $(0 < \alpha \leq 1)$ $\uparrow$ Reward received $\uparrow$ TD Target $\uparrow$ Feceived $\uparrow$ Value of next state-action (actually taken) $\uparrow$ TD Error $\uparrow$ Current estimate

Key Point:

We use A_{t+1} actually taken by the current policy π . Hence, it is On-Policy.

4. ALGORITHM (TD-SARSA)

1. Initialize $Q(s, a)$ arbitrarily for all (s, a) .
2. Choose a policy π (e.g., ϵ -greedy) and set step-size α .
3. For each episode:
 - a) Initialize S .
 - b) Choose action A from S using policy π .
 - c) Repeat (for each step of episode):
 - i) Take action A , observe R, S' .
 - ii) Choose action A' from S' using policy π .
 - iii) Update: $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$
 - iv) $S \leftarrow S', A \leftarrow A'$
 - d) Until episode terminates.

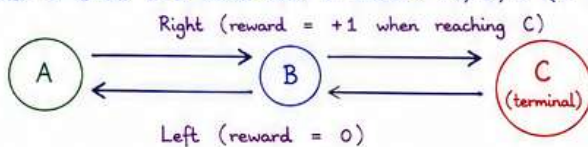
Compared to Q-learning (Off-policy), TD-SARSA uses the next action we actually follow.

Properties

- ✓ On-policy control method
- ✓ Combines ideas of Monte-Carlo (uses experience) and Dynamic Programming (bootstrapping).
- ✓ Converges to Q^π under suitable conditions.

5. SIMPLE EXAMPLE

Consider a small 1-D world with 3 states: A, B, C (C is terminal).



Parameters

$\gamma = 1, \alpha = 0.5$, ϵ -greedy policy (here we assume actions shown below are taken).

Step t	State S_t	Action A_t	Reward R_{t+1}	Next State S_{t+1}	Next Action A_{t+1}	Old $Q(S_t, A_t)$	TD Target $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$	TD Error	Updated $Q(S_t, A_t)$
1	A	Right	0	B	Right	0	$0 + 1 \cdot 0 = 0$	$0 - 0 = 0$	$0 + 0.5 \cdot 0 = 0$
2	B	Right	+1	C	—	0	$1 + 1 \cdot 0 = 1$	$1 - 0 = 1$	$0 + 0.5 \cdot 1 = 0.5$
3	A	Right	0	B	Right	0	$0 + 1 \cdot 0.5 = 0.5$	$0.5 - 0 = 0.5$	$0 + 0.5 \cdot 0.5 = 0.25$

After more episodes, $Q(A, \text{Right}) \rightarrow 0.5$ and $Q(B, \text{Right}) \rightarrow 1$ (optimal values for this problem).

6. SUMMARY

- ★ TD-SARSA learns action values from experience (sample by sample).
- ★ It updates using the next action actually taken (on-policy).
- ★ It is simple, efficient and widely used in many RL problems.

In Short

TD-SARSA = Learn + Act using same policy (π) and improve it over time.

Learn from
experience
(step by step)

SARSA ALGORITHM - SOLVED EXAMPLE

(On-policy TD control)

On-policy: We learn
and act using the
same policy.

1. SARSA UPDATE RULE

SARSA updates the action-value function $Q(s,a)$ using the transition actually taken.

$$Q(S_t, a_t) \leftarrow Q(S_t, a_t) + \alpha [r_{t+1} + \gamma Q(S_{t+1}, a_{t+1}) - Q(S_t, a_t)]$$

old estimate learning rate ($0 < \alpha \leq 1$) reward received discount factor ($0 \leq \gamma < 1$) value of next state-action (actual next action taken) TD error (difference)

Parameters used in example

- Step-size (α) = 0.5
- Discount factor (γ) = 1
- Exploration policy = ϵ -greedy ($\epsilon = 0.1$)
- Rewards: -1 for each step
+10 at goal
- All $Q(s,a)$ initialized to 0

2. ENVIRONMENT (GRID WORLD)

S (0,0)	→	↕	↔	↕	←	G (0,3) +10
(1,0)		(1,1)		(1,2)		(1,3)
(2,0)		(2,1)		(2,2)		(2,3)

- Start = S (0,0)
- Goal = G (0,3) → +10
- Obstacle = (1,3) (cannot enter)
- Each move → -1
- Episode ends when goal is reached.

↑ Up → Right
↓ Down ← Left

One episode (sample path)

(0,0) → (0,1) → (0,2) → (0,3: Goal)
Right Right Right

We will update Q values along this path using SARSA.

3. STEP-BY-STEP SARSA UPDATES

Step t	Current State S_t	Action a_t (taken)	Reward r_{t+1}	Next State S_{t+1}	Next Action a_{t+1} (chosen)	Old $Q(S_t, a_t)$	Target $r_{t+1} + \gamma Q(S_{t+1}, a_{t+1})$	TD Error (Target - Old)	Updated $Q(S_t, a_t)$
0	(0,0)	Right	-1	(0,1)	Right	0	$-1 + 1 \times 0 = -1$	$-1 - 0 = -1$	$0 + 0.5(-1) = -0.5$
1	(0,1)	Right	-1	(0,2)	Right	0	$-1 + 1 \times 0 = -1$	$-1 - 0 = -1$	$0 + 0.5(-1) = -0.5$
2	(0,2)	Right	+10	(0,3) Goal	— (terminal)	0	$10 + 1 \times 0 = 10$	$10 - 0 = 10$	$0 + 0.5(10) = 5$

At the goal state, $Q(G, \cdot) = 0$ because episode terminates. So target = reward only.

After this episode:

- $Q(0,0, \text{Right}) = -0.5$
- $Q(0,1, \text{Right}) = -0.5$
- $Q(0,2, \text{Right}) = 5$

More episodes will further improve these values.
The best action from (0,2) is Right (value 5).

4. Q-TABLE AFTER FIRST EPISODE (Only Right actions shown)

State	Up	Down	Left	Right
(0,0)	0	0	0	-0.5
(0,1)	0	0	0	-0.5
(0,2)	0	0	0	5
(0,3) Goal	0	0	0	0
(1,3) Blocked	—	—	—	—
Other states	0	0	0	0

Interpretation:

- Negative values mean it costs steps (-1 per move).
- Positive value at (0,2) → Right because it leads to goal (+10).
- SARSA learns "on-policy" because it uses the next action a_{t+1} that is actually chosen by policy.

5. SARSA ALGORITHM (SUMMARY)

1. Initialize $Q(s,a) = 0$ for all (s,a) .
2. Choose a policy π (e.g., ϵ -greedy).
3. Initialize S (start state).
4. Choose action A from S using π .
5. Repeat (for each step of episode):
 - a. Take action A, observe R, S' .
 - b. Choose next action A' from S' using π .
 - c. Update: $Q(S,A) \leftarrow Q(S,A) + \alpha [R + \gamma Q(S',A') - Q(S,A)]$
 - d. $S \leftarrow S'$, $A \leftarrow A'$.
6. Until S is terminal (goal).

Key Takeaways

- ✓ SARSA learns from experience (no model needed).
- ✓ Uses the next action actually taken → On-policy.
- ✓ Balances exploration and exploitation.
- ✓ Converges to optimal action-value function under suitable conditions.



On-policy:
We learn and act
using the same
policy.

SARSA ALGORITHM - SOLVED EXAMPLE

(On-policy TD Control)

Idea: Learn the value
of action we actually
take, following the
current policy.

1. UPDATE RULE (SARSA)

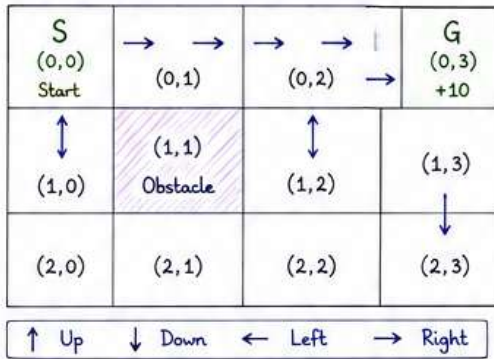
$$Q(S_t, a_t) \leftarrow Q(S_t, a_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, a_{t+1}) - Q(S_t, a_t)]$$

old estimate learning rate ($0 < \alpha \leq 1$) reward received discount factor ($0 \leq \gamma < 1$) value of next state-action (next action actually taken) TD error (difference)

Parameters for this example

- Step-size $\alpha = 0.5$
- Discount factor $\gamma = 1$
- Policy ϵ -greedy, $\epsilon = 0.2$
- Rewards: -1 for each step
+10 at goal
- All $Q(s, a)$ initialized to 0

2. ENVIRONMENT (GRID WORLD)

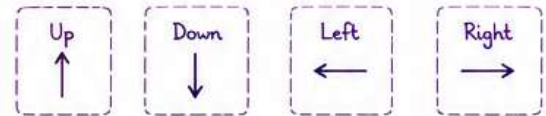


- Start = (0,0)
- Goal = (0,3) → +10
- Obstacle = (1,1) (cannot enter)
- Hitting the boundary keeps the agent in the same state.
- Episode ends when goal is reached.

One episode (sample path followed)

(0,0) → (0,1) → (0,2) → (0,3) Goal
Right Right Right

Actions available at any state



3. STEP-BY-STEP SARSA UPDATES

Step t	Current State S_t	Action A_t (taken)	Reward R_{t+1}	Next State S_{t+1}	Next Action A_{t+1} (chosen)	Old $Q(S_t, A_t)$	Target $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$	TD Error (Target - Old)	Updated $Q(S_t, A_t)$
0	(0,0)	Right	-1	(0,1)	Right	0	$-1 + 1 \times 0 = -1$	$-1 - 0 = -1$	$0 + 0.5(-1) = -0.5$
1	(0,1)	Right	-1	(0,2)	Right	0	$-1 + 1 \times 0 = -1$	$-1 - 0 = -1$	$0 + 0.5(-1) = -0.5$
2	(0,2)	Right	+10	(0,3) Goal	— (terminal)	0	$10 + 1 \times 0 = 10$	$10 - 0 = 10$	$0 + 0.5(10) = 5$

Note: At the goal (terminal) state, $Q(S_{\text{terminal}}, a) = 0$ for all actions, so target = reward only.

After episode 1:

- $Q((0,0), \text{Right}) = -0.5$
- $Q((0,1), \text{Right}) = -0.5$
- $Q((0,2), \text{Right}) = 5$

More episodes will propagate these values backward and improve them.

4. Q-TABLE AFTER FIRST EPISODE

State	$Q(s, \text{Up})$	$Q(s, \text{Down})$	$Q(s, \text{Left})$	$Q(s, \text{Right})$
(0,0)	0	0	0	-0.5
(0,1)	0	0	0	-0.5
(0,2)	0	0	0	5
(0,3) (Goal)	0	0	0	0
(1,1) (Obstacle)	—	—	—	—
All other states	0	0	0	0

Interpretation

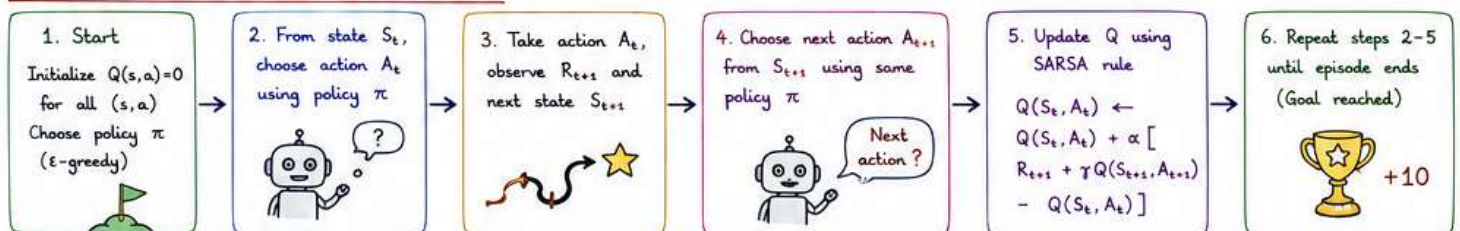
- The best action so far at (0,2) is Right with value 5 (leads to goal).
- At (0,0) and (0,1) the value is negative because a step cost (-1) is involved.
- As more episodes are run, values converge to the optimal action values under the current policy.

Typical value progression (at state (0,0), action Right)

Episode	$Q((0,0), \text{Right})$
1	-0.5
2	-0.75
5	-1.31
10	-1.73
20	-2.24
...	...

(Values converge)

5. SUMMARY OF SARSA (ON-POLICY)



Key Takeaways

- ✓ SARSA is an on-policy [TD control] algorithm.
- ✓ It learns action values for the actions actually taken.
- ✓ It uses experience (sample episodes) and updates step by step.
- ✓ Converges to optimal action-value function under suitable conditions.

SARSA vs Q-learning

- SARSA (On-policy): Update uses next action actually taken A_{t+1} (includes exploration).
- Q-learning (Off-policy): Update uses best next action $\max_{a'} Q(S_{t+1}, a')$ (greedy target).

Remember

On-policy =
Learn & Act with
same policy π



On-policy:
We learn and act
using the same
policy.

SARSA ALGORITHM - SOLVED EXAMPLE

(On-policy TD Control)

Idea: Learn the value
of action we actually
take, following the
current policy.

1. UPDATE RULE (SARSA)

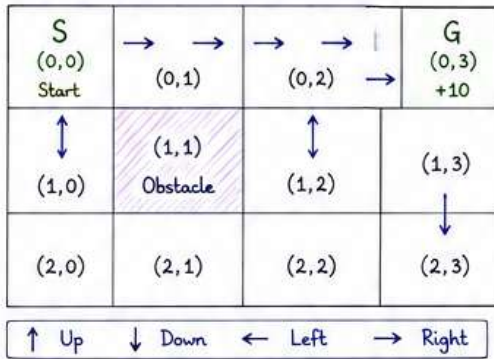
$$Q(S_t, a_t) \leftarrow Q(S_t, a_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, a_{t+1}) - Q(S_t, a_t)]$$

old estimate learning rate ($0 < \alpha \leq 1$) reward received discount factor ($0 \leq \gamma < 1$) value of next state-action (next action actually taken) TD error (difference)

Parameters for this example

- Step-size $\alpha = 0.5$
- Discount factor $\gamma = 1$
- Policy ϵ -greedy, $\epsilon = 0.2$
- Rewards: -1 for each step
+10 at goal
- All $Q(s, a)$ initialized to 0

2. ENVIRONMENT (GRID WORLD)

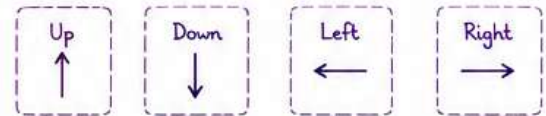


- Start = (0,0)
- Goal = (0,3) → +10
- Obstacle = (1,1) (cannot enter)
- Hitting the boundary keeps the agent in the same state.
- Episode ends when goal is reached.

One episode (sample path followed)

(0,0) → (0,1) → (0,2) → (0,3) Goal
Right Right Right

Actions available at any state



3. STEP-BY-STEP SARSA UPDATES

Step t	Current State S_t	Action A_t (taken)	Reward R_{t+1}	Next State S_{t+1}	Next Action A_{t+1} (chosen)	Old $Q(S_t, A_t)$	Target $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$	TD Error (Target - Old)	Updated $Q(S_t, A_t)$
0	(0,0)	Right	-1	(0,1)	Right	0	$-1 + 1 \times 0 = -1$	$-1 - 0 = -1$	$0 + 0.5(-1) = -0.5$
1	(0,1)	Right	-1	(0,2)	Right	0	$-1 + 1 \times 0 = -1$	$-1 - 0 = -1$	$0 + 0.5(-1) = -0.5$
2	(0,2)	Right	+10	(0,3) Goal	— (terminal)	0	$10 + 1 \times 0 = 10$	$10 - 0 = 10$	$0 + 0.5(10) = 5$

Note: At the goal (terminal) state, $Q(S_{\text{terminal}}, a) = 0$ for all actions, so target = reward only.

After episode 1:

- $Q((0,0), \text{Right}) = -0.5$
- $Q((0,1), \text{Right}) = -0.5$
- $Q((0,2), \text{Right}) = 5$

More episodes will propagate these values backward and improve them.

4. Q-TABLE AFTER FIRST EPISODE

State	$Q(s, \text{Up})$	$Q(s, \text{Down})$	$Q(s, \text{Left})$	$Q(s, \text{Right})$
(0,0)	0	0	0	-0.5
(0,1)	0	0	0	-0.5
(0,2)	0	0	0	5
(0,3) (Goal)	0	0	0	0
(1,1) (Obstacle)	—	—	—	—
All other states	0	0	0	0

Interpretation

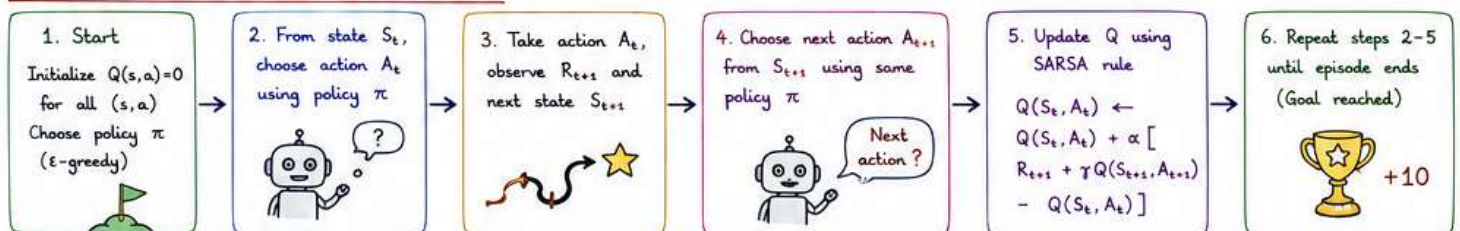
- The best action so far at (0,2) is Right with value 5 (leads to goal).
- At (0,0) and (0,1) the value is negative because a step cost (-1) is involved.
- As more episodes are run, values converge to the optimal action values under the current policy.

Typical value progression (at state (0,0), action Right)

Episode	$Q((0,0), \text{Right})$
1	-0.5
2	-0.75
5	-1.31
10	-1.73
20	-2.24
...	...

(Values converge)

5. SUMMARY OF SARSA (ON-POLICY)



Key Takeaways

- ✓ SARSA is an on-policy [TD control] algorithm.
- ✓ It learns action values for the actions actually taken.
- ✓ It uses experience (sample episodes) and updates step by step.
- ✓ Converges to optimal action-value function under suitable conditions.

SARSA vs Q-learning

- SARSA (On-policy): Update uses next action actually taken A_{t+1} (includes exploration).
- Q-learning (Off-policy): Update uses best next action $\max_{a'} Q(S_{t+1}, a')$ (greedy target).

Remember

On-policy =
Learn & Act with
same policy π



Goal: Learn optimal action-value function from experience!

TD-Q-LEARNING (OFF-POLICY TD CONTROL)

Off-policy: We learn about the best policy (greedy) while following a different behavior policy (exploring).

1. WHAT IS TD-Q-LEARNING ?

- TD-Q-learning is an **off-policy** Temporal-Difference (TD) control algorithm.
- It learns the **optimal action-value function** $Q^*(s, a)$ directly from interaction, without needing a model of the environment.
- It combines ideas of Q-learning (off-policy) and TD learning (bootstrapping).
- It updates Q-values using the next state's maximum future value.

2. TD-Q-LEARNING UPDATE RULE

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

↑ Old estimate ↑ Learning rate ($0 < \alpha \leq 1$) ↑ Reward received ↑ Discount factor ($0 \leq \gamma < 1$) ↑ Best future action value in next state s_{t+1} ↑ TD Error (difference)

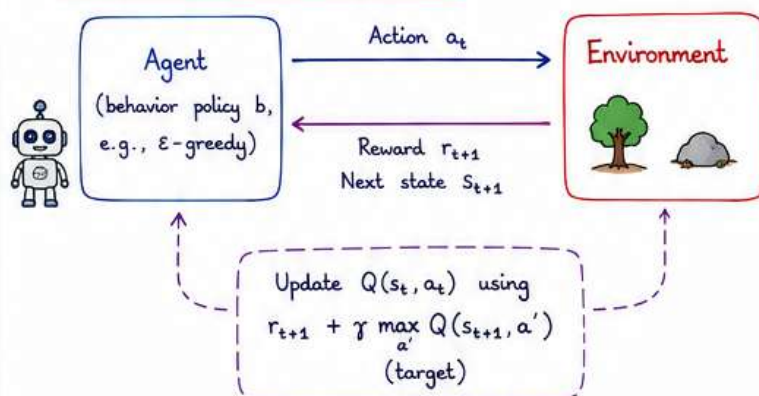
Key idea: We bootstrap from the maximum next action value, irrespective of the action actually taken.

3. ALGORITHM (TD-Q-LEARNING)

- Initialize $Q(s, a)$ arbitrarily (e.g., all zeros).
- Choose behavior policy b (e.g., ϵ -greedy) for exploration.
- For each episode:
 - Initialize state s .
 - Choose action a using behavior policy b .
 - Take action a , observe reward r and next state s' .
 - Update $Q(s, a)$ using the TD-Q update rule.
 - Set $s \leftarrow s'$.
 - Repeat steps b-e until terminal state.
- After many episodes, derive the greedy policy:

$$\pi^*(s) = \arg \max_a Q(s, a)$$

4. HOW IT WORKS (ONE STEP)

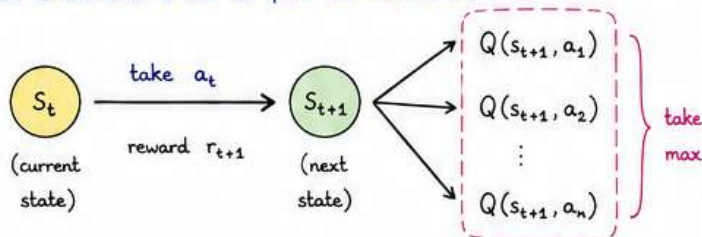


5. OFF-POLICY vs ON-POLICY

	Off-Policy (TD-Q-Learning)	On-Policy (e.g., SARSA)
	Learns about the optimal policy (π^*).	Learns about the policy being followed.
	Update uses $\max_{a'} Q(s', a')$ (best next action).	Update uses $Q(s', a')$ where a' is the action actually taken.
	More optimistic (may take risks).	More conservative (safer).
	Usually faster convergence.	Usually more stable.

6. INTUITION

TD-Q-learning always looks ahead and asks: "What is the best I can do from the next state?" Look at all possible actions in s_{t+1}



Use the **maximum** of these values in the update!

7. TD ERROR (WHAT IS BEING LEARNED?)

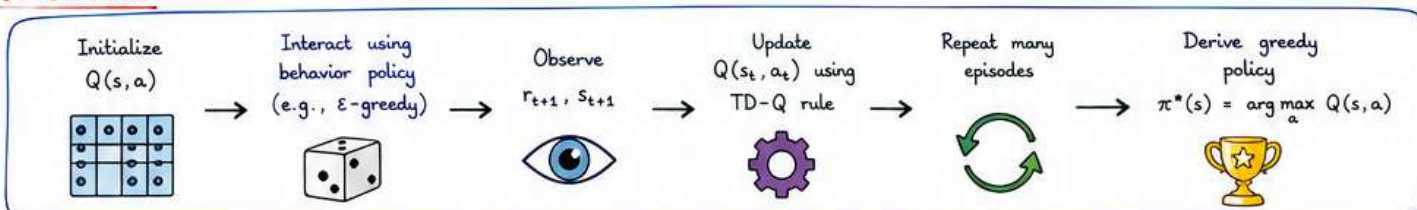
$$\text{TD Error} = \text{Target} - \text{Current Estimate}$$

Target = $r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$
(best future reward estimate)

Current Estimate = $Q(s_t, a_t)$
(our present guess)

We adjust $Q(s_t, a_t)$ in the direction of the TD error.

9. SUMMARY



TD-Q-Learning is an **off-policy** TD control method that learns the best action-values from experience by bootstrapping with the **maximum** future value.



Learn the optimal action values from experience!

TD-Q-LEARNING (OFF-POLICY TD CONTROL)

Off-policy: We learn the optimal policy (greedy) while behaving differently (exploring).

1. WHAT IS TD-Q-LEARNING?

- TD-Q-learning is an off-policy Temporal-Difference control algorithm.
- It learns the optimal action-value function $Q^*(s,a)$ directly, without following the greedy policy while exploring.
- It is model-free and updates Q-values using the next state's maximum action value.

2. TD-Q-LEARNING UPDATE RULE

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

↑ Old estimate
 ↑ Learning rate $(0 < \alpha \leq 1)$
 ↑ Reward received
 ↑ Discount factor $(0 \leq \gamma < 1)$
 ↑ Best future action value in next state s_{t+1}
 ↑ TD Error (difference)

Key idea: We bootstrap from the best next action value, irrespective of the action actually taken.

3. ALGORITHM (TD-Q-LEARNING)

- Initialize $Q(s,a)$ arbitrarily (e.g., all zeros).
- Choose behavior policy (e.g., ϵ -greedy) for exploration.
- For each episode:
 - Initialize state s .
 - Choose action a using behavior policy.
 - Take action a , observe reward r and next state s' .
 - Update $Q(s,a)$ using the TD-Q update rule.
 - Set $s \leftarrow s'$.
 - Repeat steps b - e until terminal state.
- After many episodes, derive the greedy policy: $\pi^*(s) = \arg \max_a Q(s,a)$

4. ON-POLICY vs OFF-POLICY

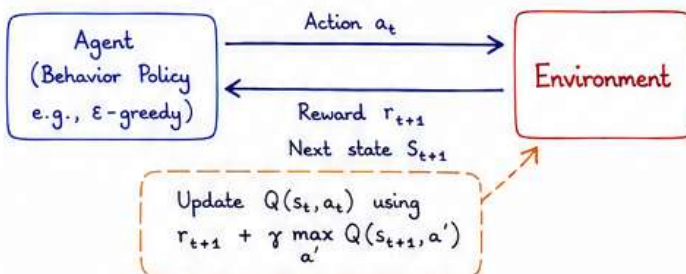
On-Policy (e.g., SARSA)	Off-Policy (Q-Learning)
Learns the value of the policy that is being followed.	Learns the optimal policy (independent of behavior policy).
Update uses actual next action taken.	Update uses max over all possible next actions.
Generally more cautious.	Generally faster convergence.
Example: SARSA	Example: Q-Learning

5. ADVANTAGES

- Converges to optimal action-value function Q^* under suitable conditions.
- Off-policy - can learn optimal policy while exploring.
- Simple and effective.
- Widely used in many RL problems.



6. HOW IT WORKS (ONE STEP)



When does it converge?

If every state-action pair is visited infinitely often and the learning rate satisfies Robbins-Monro conditions ($\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$), Q-learning converges to Q^* .

7. SIMPLE EXAMPLE (GRID WORLD)

(0,0)	(0,1)	(0,2)	GOAL (+10)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)

Blocked cell

Start state: (0,0)
Goal state: (0,3) with +10
All other moves: -1
Actions: Up, Down, Left, Right
If hits boundary or blocked cell → stay in same state and get -1.

One episode (sample path)

Step t	State s_t	Action a_t	Reward r_{t+1}	Next State s_{t+1}	$\max_{a'} Q(s_{t+1}, a')$ (assume current)	TD Target $r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$	Updated $Q(s_t, a_t)$ ($\alpha = 0.5, \gamma = 1$)
0	(0,0)	Right	-1	(0,1)	0	-1	$0 + 0.5(-1 - 0) = -0.5$
1	(0,1)	Right	-1	(0,2)	0	-1	$0 + 0.5(-1 - 0) = -0.5$
2	(0,2)	Right	+10	(0,3)	0 (terminal)	+10	$0 + 0.5(10 - 0) = 5$

Optimal Policy (one possible)

→ → → (towards goal)
↓ (down around the block)
→ → ↑

After many episodes, Q-values propagate back and the agent learns the best path to the goal.



8. SUMMARY



- ★ TD-Q-learning learns $Q^*(s,a)$ using TD update with max over actions.
- ★ It is off-policy - learns optimal policy while exploring.
- ★ Converges to optimal values under suitable conditions.
- ★ Simple, model-free and powerful for many tasks.

In Short

TD-Q-Learning = Learn optimal action-values using experience and the best possible future value.



Goal:
Learn optimal action-value function $Q^*(s,a)$ to act optimally in the future.

TD-Q-LEARNING (Off-Policy TD Control)

Off-policy: Learn the best policy (greedy) while following a different behavior policy (exploring).

1. TD-Q-LEARNING UPDATE RULE

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

↑ Old estimate ↑ Learning rate ($0 < \alpha \leq 1$) ↑ Reward received ↑ Discount factor ($0 \leq \gamma < 1$) ↑ Best future action value in next state s_{t+1} ↑ TD Error (difference)

Key idea:

We bootstrap from the maximum Q-value in the next state, irrespective of the action actually taken.

2. EXAMPLE ENVIRONMENT

(0,0) S Start	(0,1)	(0,2)	(0,3) G Goal +10
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)

- Start state: (0,0)
- Goal state: (0,3) with reward +10
- Obstacle: (1,1)
- Reward: -1 for every move until the goal is reached.
- Episode ends when goal is reached.

One episode (sample path taken)

(0,0) → (0,1) → (0,2) → (0,3)
Right Right Right Goal

↑ Up ↓ Down ← Left → Right

■ = Obstacle (cannot enter)

Parameters used in this example

α (learning rate) = 0.5
 γ (discount factor) = 1.0 (for simplicity)
 All $Q(s,a)$ initialized to 0

3. STEP-BY-STEP TD-Q-LEARNING UPDATES

Step t	Current State s_t	Action Taken a_t	Reward r_{t+1}	Next State s_{t+1}	All Q-values in next state s_{t+1}				$\max_{a'} Q(s_{t+1}, a')$	TD Target $r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$	Old $Q(s_t, a_t)$	TD Error (Target - Old)	New $Q(s_t, a_t)$
					Up	Down	Left	Right					
0	(0,0)	Right	-1	(0,1)	0	0	0	0	0	$-1 + 1 \times 0 = -1$	0	$-1 - 0 = -1$	$0 + 0.5(-1) = -0.5$
1	(0,1)	Right	+1	(0,2)	0	0	0	0	0	-1	0	$-1 - 0 = -1$	-0.5
2	(0,2)	Right	+10	(0,3) (Goal)	0	0	0	0	0	$10 + 1 \times 0 = 10$	0	$10 - 0 = 10$	5

At the goal (terminal) state (0,3), all Q-values are 0 because there are no future rewards beyond the goal.

After Episode 1:

$Q((0,0), \text{Right}) = -0.5$
 $Q((0,1), \text{Right}) = -0.5$
 $Q((0,2), \text{Right}) = 5$

More episodes:

Values will propagate backward and converge to optimal values.

4. Q-TABLE AFTER EPISODE 1

State	Q-values			
	Up	Down	Left	Right
(0,0)	0	0	0	-0.5
(0,1)	0	0	0	-0.5
(0,2)	0	0	0	5
(0,3) Goal	0	0	0	0
(1,1) Obstacle	-	-	-	-
Other states	0	0	0	0

Interpretation:

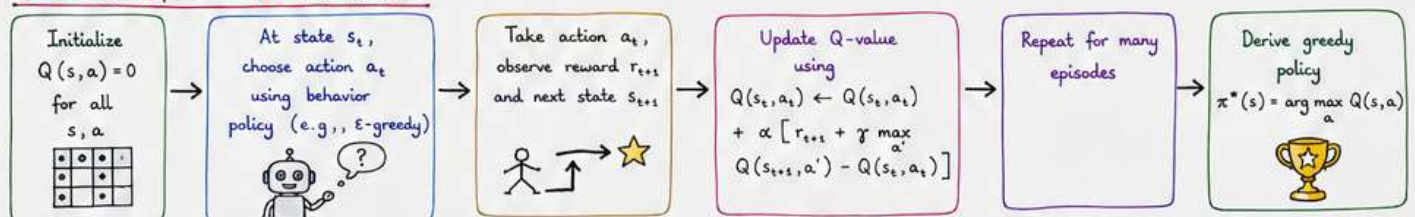
- Best action from (0,2) is Right (value 5) because it leads to the goal with +10.
- From (0,1) and (0,0), values are negative because each step has a cost (-1).
- With more episodes, the values will improve and the optimal policy (always Right) will emerge.

Typical value progression for $Q((0,0), \text{Right})$

Episode	$Q((0,0), \text{Right})$
0 (init)	0
1	-0.5
2	-0.75
5	-1.31
10	-1.73
20	-2.24
...	...

(Converges to optimal value)

5. SUMMARY OF TD-Q-LEARNING



Key Takeaways

- ✓ TD-Q-Learning is an off-policy TD control algorithm.
- ✓ It learns the optimal action-value function $Q^*(s,a)$ while following an exploratory behavior policy.
- ✓ It uses the maximum Q-value in the next state (greedy target) in the update.
- ✓ Simple, model-free and widely used in RL.

When does it converge?

Under suitable conditions (sufficient exploration, decaying learning rate or small constant α), TD-Q-learning converges to Q^* .



Off-policy: Learn the best policy (greedy) while following a different behavior policy (exploration).

TD-Q-LEARNING (Off-Policy TD Control)

Goal: Learn optimal action-value function $Q^*(s,a)$ to act optimally in the future.

1. UPDATE RULE

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

$\uparrow$ Old estimate $\uparrow$ Learning rate ($0 < \alpha \leq 1$) $\uparrow$ Reward received $\uparrow$ Discount factor ($0 \leq \gamma < 1$) $\uparrow$ Best future action value in next state s_{t+1} $\uparrow$ TD Error (difference)

Parameters for this example

$\alpha = 0.5$ (learning rate)
 $\gamma = 0.9$ (discount factor)
 All $Q(s,a)$ initially = 0
 Behavior policy: ϵ -greedy ($\epsilon=0.2$ for exploration)

3. ONE EPISODE (SAMPLE PATH)

$(2,0) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (1,2) \times \rightarrow (1,1) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3)$
 Right Right Up (blocked) Up Right Goal

Actual path followed by the agent (with exploration):

$(2,0) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (1,1) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3)$

At $(1,2)$ the agent tries Up but hits obstacle, so it stays in $(2,2)$ and then goes Left to $(1,1)$.

2. EXAMPLE ENVIRONMENT

(0,0)	(0,1)	(0,2)	(0,3) Goal +10
(1,0)	(1,1)		(1,3)
(2,0)	(2,1)	(2,2)	(2,3)

= Obstacle (cannot enter)

$\uparrow$ Up $\downarrow$ Down $\leftarrow$ Left $\rightarrow$ Right

- Start state: $(2,0)$
- Goal state: $(0,3)$ with reward +10
- Obstacle: $(1,2)$ cannot be entered
- Reward: -1 for every move until goal
- Episode ends when goal is reached.

4. STEP-BY-STEP TD-Q-LEARNING UPDATES

Step t	Current State s_t	Action Taken a_t	Reward r_{t+1}	Next State s_{t+1}	$\max_{a'} Q(s_{t+1}, a')$ (Before Update)	$Q(s_t, a_t)$ (Before Update)	TD Target $r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$	TD Error (Target - Old)	New $Q(s_t, a_t)$ (Updated)
0	(2,0)	Right	-1	(2,1)	0	0	$-1 + 0.9 \times 0 = -1$	$-1 - 0 = -1$	$0 + 0.5(-1) = -0.5$
1	(2,1)	Right	-1	(2,2)	0	0	$-1 + 0.9 \times 0 = -1$	$-1 - 0 = -1$	-0.5
2	(2,2)	Left	-1	(1,1)	0	0	$-1 + 0.9 \times 0 = -1$	$-1 - 0 = -1$	-0.5
3	(1,1)	Up	-1	(0,1)	0	0	$-1 + 0.9 \times 0 = -1$	$-1 - 0 = -1$	-0.5
4	(0,1)	Right	-1	(0,2)	0	0	$-1 + 0.9 \times 0 = -1$	$-1 - 0 = -1$	-0.5
5	(0,2)	Right	+10	(0,3) Goal	0	0	$10 + 0.9 \times 0 = 10$	$10 - 0 = 10$	$0 + 0.5(10) = 5$

Note: At goal state $(0,3)$, $\max_{a'} Q(0,3, a') = 0$ because there are no future rewards after reaching the goal.

After Episode 1 - Updated Q-values (only visited state-action pairs)

$Q((2,0), \text{Right}) = -0.5$ $Q((2,1), \text{Right}) = -0.5$ $Q((2,2), \text{Left}) = -0.5$
 $Q((1,1), \text{Up}) = -0.5$ $Q((0,1), \text{Right}) = -0.5$ $Q((0,2), \text{Right}) = 5$

5. Q-TABLE AFTER EPISODE 1 (Showing max Q for right action in each state)

(0,0)	(0,1)	(0,2)	(0,3) Goal
0	-0.5	5	0
(1,0)	(1,1)	(1,2) Obstacle	(1,3)
0	-0.5	X	0
(2,0)	(2,1)	(2,2)	(2,3)
-0.5	-0.5	-0.5	0

Interpretation:

- ✓ The agent has learned that moving Right from $(0,2)$ is very good (value 5) because it leads to the goal.
- ✓ Negative values (-0.5) indicate it costs 1 step to reach better states.
- ✓ With more episodes, these values will improve and converge to optimal Q^* .

6. SUMMARY OF ALGORITHM

- ✓ Initialize $Q(s,a)$ arbitrarily (often 0).
- ✓ For each episode:
 - ✓ At state s , choose action a using behavior policy (ϵ -greedy).
 - ✓ Take action a , observe r and next state s' .
 - ✓ Update $Q(s,a)$ using TD-Q update rule.
 - ✓ Set $s \leftarrow s'$ and repeat until goal.
- ✓ After many episodes, derive greedy policy $\pi^*(s) = \arg \max_a Q(s,a)$.

7. KEY TAKEAWAYS

- ☆ TD-Q-Learning is an off-policy TD control algorithm.
- ☆ It learns optimal action-value function using the best next action value (max).
- ☆ It allows exploration during learning but results in a greedy (optimal) policy.
- ☆ Simple, model-free and widely used in many RL problems.



Learn $\rightarrow$ Update $\rightarrow$
Repeat $\rightarrow$ Optimal!